DevOps Training Program

Lab 9 — Emergency Recovery

Recovering Access to EC2 When the PEM Key is Lost

⚠️ Real-World Scenario

Your EC2 instance is running with important data. You deleted or lost the .pem key file. You cannot SSH in. The instance must NOT be terminated.

This lab teaches you TWO official recovery methods.

⏱️ Duration	📊 Difficulty
3–4 Hours	Advanced ⭐⭐⭐

The Golden Rule: You CANNOT Recover a Lost PEM Key

FACT	AWS does NOT store your private key (.pem file) anywhere. Once you delete it from your computer, it is gone forever — not even AWS Support can give it back. The key pair shown in EC2 console is only the PUBLIC key (stored on the instance). The private key was yours to keep.

But here is the good news: you can still recover ACCESS to your instance and its DATA. There are two methods:

METHOD 1	METHOD 2
EBS Volume Swap	**AWS Systems Manager (SSM)**
Detach root volume → attach to rescue instance → add new public key → reattach	Use SSM Session Manager for browser-based shell — no key needed

WHICH METHOD?	Use Method 1 (EBS Swap) when the SSM agent is NOT installed on the instance. Use Method 2 (SSM) when the SSM agent IS running and an IAM role is attached — much faster. This lab teaches BOTH.

Pre-Lab Setup — Simulate the Problem

First, let's create the problem we will solve — a locked EC2 instance.

STEP 1 Launch a Fresh EC2 Instance (the 'Locked' Instance)

1. EC2 Console → Launch Instance
2. Name: locked-instance | AMI: Amazon Linux 2023 | Type: t2.micro
3. Key pair: Create a new one called temp-key → Download it → then IMMEDIATELY delete it from your downloads folder to simulate losing it
4. Security Group: Allow SSH (22) from My IP
5. Launch the instance and note its Instance ID and Availability Zone (AZ) from the EC2 console

SIMULATE LOSS Once you delete temp-key.pem from your computer, you cannot SSH in. You now have a running instance you're locked out of. This is the exact real-world problem.

METHOD 1

EBS Volume Swap Recovery

Works even if SSM is not installed. Takes ~15–20 minutes.

How It Works — The Core Idea

Think of an EC2 instance like a computer, and its EBS root volume like a hard drive. The authorized_keys file (which stores which public keys can log in) lives on that hard drive. If we take that hard drive and plug it into a working computer (rescue instance), we can edit the authorized_keys file to add a new key. Then we plug the hard drive back into the original machine and log in with the new key.

STEP 2 Generate a NEW Key Pair — Your Replacement Key

6. EC2 Console → Key Pairs → Create key pair
7. Name: recovery-key | Type: RSA | Format: .pem → Create key pair
8. Save recovery-key.pem safely to your computer
9. Now extract its public key — you will need this in a few steps:

```
chmod 400 recovery-key.pem
ssh-keygen -y -f recovery-key.pem
```

Copy the entire output (starts with ssh-rsa AAAA...). This is the PUBLIC key you will inject into the locked instance.

STEP 3 Launch a Rescue EC2 Instance

10. EC2 Console → Launch Instance
11. Name: rescue-instance | AMI: Amazon Linux 2023 | Type: t2.micro
12. Key pair: recovery-key (the new key we just created)
13. IMPORTANT: Place this rescue instance in the SAME Availability Zone as your locked instance (e.g., both in us-east-1a)
14. Security Group: Allow SSH (22) from My IP → Launch

WHY SAME AZ? EBS volumes can only be attached to instances in the same Availability Zone. If your locked instance is in us-east-1a, the rescue instance must also be in us-east-1a — not us-east-1b.

STEP 4 Stop the Locked Instance (NOT Terminate)

CRITICAL STOP — do NOT TERMINATE. Stop preserves the EBS volume and all data.

Terminate deletes everything. Double-check before clicking.

15. EC2 Console → select locked-instance → Instance State → Stop instance → Confirm
16. Wait for Instance State to show: Stopped

STEP 5 Detach the Root EBS Volume from the Locked Instance

17. EC2 Console → left menu → Elastic Block Store → Volumes
18. Find the volume attached to locked-instance — it will show the Instance ID in the 'Attached instances' column
19. Right-click the volume → Detach Volume → Confirm
20. Wait for the State to change from in-use to available
21. Note down the Volume ID (e.g., vol-0abc123...)

STEP 6 Attach the Volume to the Rescue Instance

22. Right-click the same volume → Attach Volume
23. Instance: select rescue-instance from the dropdown
24. Device name: /dev/xvdf (leave the suggested name, do NOT use /dev/xvda which is the rescue instance's own root)
25. Click Attach Volume → wait for State: in-use

STEP 7 SSH into the Rescue Instance & Mount the Volume

```
chmod 400 recovery-key.pem
ssh -i recovery-key.pem ec2-user@<rescue-instance-public-ip>
```

Once inside the rescue instance, find and mount the attached volume:

```
lsblk    # You should see xvdf listed — this is the locked instance's drive
sudo mkdir /mnt/recovery
sudo mount /dev/xvdf1 /mnt/recovery
# If above fails, try without the partition number:
# sudo mount /dev/xvdf /mnt/recovery
ls /mnt/recovery    # Should look like a Linux root filesystem: bin, etc, home,
var...
```

STEP 8 Inject Your New Public Key

Navigate to the authorized_keys file inside the mounted volume and add your new public key:

```
cat /mnt/recovery/home/ec2-user/.ssh/authorized_keys
# You'll see the OLD public key (the one paired with your lost .pem)
```

Now add your new public key:

```
echo 'PASTE-YOUR-PUBLIC-KEY-HERE' | sudo tee -a /mnt/recovery/home/ec2-user/.ssh/authorized_keys
```

Replace PASTE-YOUR-PUBLIC-KEY-HERE with the full ssh-rsa AAAA... output you copied in Step 2.

```
# Verify both keys are now in the file:
cat /mnt/recovery/home/ec2-user/.ssh/authorized_keys
```

TIP You now have TWO keys in authorized_keys: the old one (harmless since no one has the private key anymore) and your new recovery-key. Some admins remove the old one for cleanliness — optional.

STEP 9 Unmount, Detach & Reattach to Locked Instance

Back on the rescue instance, unmount the volume cleanly:

```
sudo umount /mnt/recovery
```

In EC2 Console → Volumes → select the volume → Detach Volume → Confirm

Now reattach to the original locked instance:

26. Right-click the volume → Attach Volume
27. Instance: locked-instance
28. Device name: /dev/xvda (the ROOT device — very important, this is where it originally was)
29. Click Attach Volume

STEP 10 Start the Locked Instance & Log In with New Key

30. EC2 Console → select locked-instance → Instance State → Start instance
31. Wait for Instance State: Running and Status checks: 2/2 passed
32. Note the new Public IP (it may have changed after stop/start)

Now SSH in using your recovery key:

```
ssh -i recovery-key.pem ec2-user@<locked-instance-new-public-ip>
```

If you see the shell prompt — congratulations! You have recovered access to the locked instance without terminating it or losing any data.

SUCCESS You are now logged into the original locked instance using the new recovery-key.pem. All data, files, and configurations on the instance are intact.

METHOD 2

AWS Systems Manager (SSM) — No Key Needed

Faster method. Requires SSM Agent (pre-installed on Amazon Linux 2023) + IAM Role attached to the instance.

STEP A — Check If SSM Agent Is Running on the Locked Instance

Amazon Linux 2023 (and Amazon Linux 2) come with the SSM agent pre-installed. But it only works if an IAM role with SSM permissions is attached to the instance.

Go to: AWS Console → Systems Manager → Fleet Manager

If your locked instance appears in the list — SSM is active and Method 2 will work. If it's not listed, use Method 1.

STEP B — Create and Attach an IAM Role for SSM (if not already done)

33. IAM Console → Roles → Create Role → AWS Service → EC2 → Next
34. Search for and attach the policy: AmazonSSMManagedInstanceCore
35. Role name: ec2-ssm-role → Create Role
36. EC2 Console → select locked-instance → Actions → Security → Modify IAM Role
37. Select ec2-ssm-role → Update IAM Role
38. Wait 1–2 minutes → refresh Fleet Manager — the instance should appear

STEP C — Open a Session Manager Shell — No PEM Needed

39. AWS Console → Systems Manager → Session Manager → Start Session
40. Select your locked-instance from the list → Start Session
41. A browser-based terminal opens directly into the instance — no SSH, no key needed

```
whoami    # Confirm you are in as ssm-user or ec2-user
sudo su - ec2-user    # Switch to ec2-user if needed
```

STEP D — Add Your New Public Key via the SSM Shell

Now that you're inside, generate a new key pair locally and add the public key:

```
# On your LOCAL machine — generate a new key pair:
ssh-keygen -t rsa -b 4096 -f ~/.ssh/new-recovery-key
```

```
cat ~/.ssh/new-recovery-key.pub    # Copy this output
```

In the SSM browser shell:

```
echo 'PASTE-PUBLIC-KEY-OUTPUT-HERE' >> ~/.ssh/authorized_keys
cat ~/.ssh/authorized_keys    # Verify it was added
```

Now close the SSM session and SSH normally from your terminal:

```
ssh -i ~/.ssh/new-recovery-key ec2-user@<instance-public-ip>
```

Method Comparison — At a Glance

Comparison Point	Method 1: EBS Swap	Method 2: SSM
SSM Agent required?	No	Yes (pre-installed on Amazon Linux 2023)
IAM Role required?	No	Yes (AmazonSSMManagedInstanceCore)
Instance must be running?	No — Stop it first	Yes — must be running
Needs a rescue instance?	Yes	No
Time to complete	15–20 minutes	5–10 minutes
Works on ALL AMIs?	Yes	Only if SSM agent installed
Instance data preserved?	Yes	Yes
New PEM key required after?	Yes — add recovery key	Yes — add via shell

Key Concepts Explained

Why Can't You Retrieve the Lost PEM?

AWS uses asymmetric key cryptography. When you create a key pair, AWS generates both keys but immediately hands you ONLY the private key (.pem) and keeps only the public key (stored in ~/.ssh/authorized_keys on the instance). There is no server-side vault storing your private key. It is mathematically impossible to derive the private key from the public key.

What Is authorized_keys?

The file /home/ec2-user/.ssh/authorized_keys on your EC2 instance holds a list of public keys that are allowed to connect. When you SSH in, your private key (.pem) proves it matches one of these public keys via a mathematical challenge. Adding a new public key here is exactly what AWS does when you create the original key pair at launch time.

What Is SSM Session Manager?

AWS Systems Manager Session Manager is a browser-based shell that connects to your EC2 instance through the AWS control plane (not over port 22/SSH). It uses the SSM agent running inside the instance to establish a secure tunnel. You need zero open inbound ports — it works even with Security Groups that block ALL traffic.

Expected Outcomes

✓	Understand why a lost PEM key cannot be recovered from AWS
✓	Stop an EC2 instance safely without losing data
✓	Detach and reattach an EBS root volume between instances
✓	Mount an EBS volume on a rescue instance and edit its filesystem
✓	Inject a new SSH public key into authorized_keys manually
✓	Recover SSH access to the original locked instance using the new key
✓	Use SSM Session Manager to access an instance with NO key required
✓	Understand the difference between Stop and Terminate and why it matters

Student Submission Requirements

Screenshot 1	EC2 Console showing locked-instance in Stopped state before EBS detach
Screenshot 2	EBS Volumes list showing the volume in 'available' state after detach
Screenshot 3	SSH terminal on rescue instance showing lsblk with xvdf + ls /mnt/recovery output
Screenshot 4	cat ~/.ssh/authorized_keys showing BOTH the old key and your new recovery key
Screenshot 5	Successful SSH into locked-instance using recovery-key.pem (show the prompt)
Screenshot 6	SSM Session Manager browser terminal open inside the instance (Method 2)
Written Answer	In your own words: why can't you retrieve a lost .pem file? (2–3 sentences)

BEST PRACTICE	After completing this lab: always store PEM keys in a secure location (a password manager like Bitwarden or 1Password, or an encrypted S3 bucket). For production, always attach an SSM role to every EC2 instance at launch — it's a lifesaver.